

Comparative analysis: Go vs Java vs Julia

Aspect	Go	Java	Julia
GC maturity	Improving steadily since Go 1.5	Very mature and stable over decades	Still evolving and relatively young
Default GC behaviour	GC is always on with minimal user config	Default is G1 GC, tunable to ZGC or others	Stop-the-world GC; minimal customisation
Tooling availability	Basic profiling tools (e.g., pprof)	Advanced tools (JVisualVM, GC logs, Flight Recorder)	Limited tools for GC inspection
Generational support	No	Yes (some collectors like G1, CMS are generational)	No (GC is not generational as of the latest version)
Real-time suitability	Good fit for real-time-ish apps	Can be tuned for low-pause-time real-time environments	Not well-suited due to higher GC pauses

identifies and frees memory occupied by objects that are no longer in use. However, unlike Java and Go, Julia's GC is stop-the-world and not generational (as of the latest stable versions). This means that during garbage collection, the entire program halts until the cleanup is finished.

The benefits:

- No need for explicit memory deallocation (like in C/C++)
- Reduced risk of memory leaks and dangling pointers
- Safer and simpler code, especially in complex numerical and scientific computations

Julia's garbage collection strategy is a stop-the-world, mark-and-sweep collector — conceptually simple but highly optimised for performance. Let's break that down:

Stop-the-world: When a garbage collection cycle runs, Julia pauses all running threads to ensure memory safety. During this pause, no Julia code executes — only the garbage collector is active.

While stop-the-world strategies can introduce latency, Julia's GC is engineered to minimise pause times for typical workloads. For many numerical computing applications, brief but infrequent pauses are acceptable trade-offs.

Mark-and-sweep algorithm: Julia's GC operates in two main phases:

- **Mark phase:** Starting from GC roots (global variables, stack frames, etc), the collector traverses all reachable objects and marks them as 'alive'.
- **Sweep phase:** Once marking is complete, the collector scans through allocated memory and frees any unmarked (unreachable) objects, returning that memory to the allocator.

This method ensures that only unreachable

objects are deallocated, preventing accidental memory reclamation of live data.

Julia's GC is particularly suited for:

- **Numerical computing:** Performing calculations and simulations with numbers.
- **Data science:** Analysing and interpreting complex datasets for insights.
- **Machine learning:** Developing algorithms that learn from data automatically.

Comparative analysis: Go vs Java vs Julia

Garbage collection isn't just a behind-the-scenes detail—it directly shapes the performance, reliability, and scalability of modern software. As we've seen, Java offers a mature, configurable GC ecosystem ideal for enterprise and large-scale systems. Go takes a modern, hands-off approach, favouring concurrency and simplicity for cloud-native applications. Julia, while less mature in its GC implementation, prioritises performance for scientific workloads and high-throughput numerics.

Each language reflects a different philosophy:

- Java empowers developers with control.
- Go minimises GC complexity for developer focus.
- Julia optimises for computational intensity, even with some GC trade-offs.

In the end, the best GC strategy is not universal—it depends on your application's real-time needs, resource constraints, and performance goals. Understanding how these languages manage memory helps you not only write better code but also choose the right tool for the job. **END** 🐧

By: Raj Patel

The author is a SaaS enthusiast and technical content creator.